

Introduction to Heaps

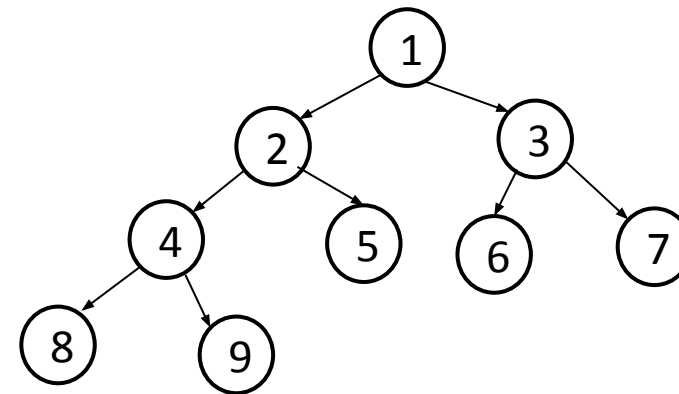
Binary Heap

A Heap is a **Binary Tree** that satisfies two additional properties:

1. Complete Binary Tree Property
2. Heap-Order Property

Complete Binary Tree Property:

If each level but the last is full, and, in the last level, all of the internal nodes are to the left of the external nodes.

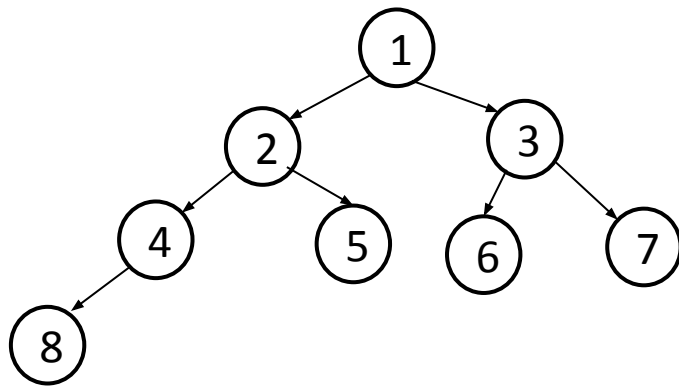


Complete Binary Tree

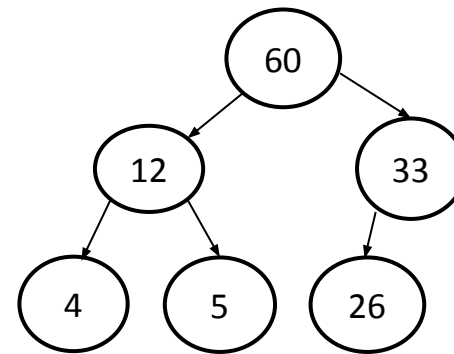
Binary Heap

Heap-Order Property: for every node v (except the root), the key stored in v is (greater than or equal) Or (less than or equal) to the key stored in v 's parent.

1. Max heap: $\text{key}(\text{parent}) \geq \text{key}(\text{child})$
2. Min heap: $\text{key}(\text{parent}) \leq \text{key}(\text{child})$



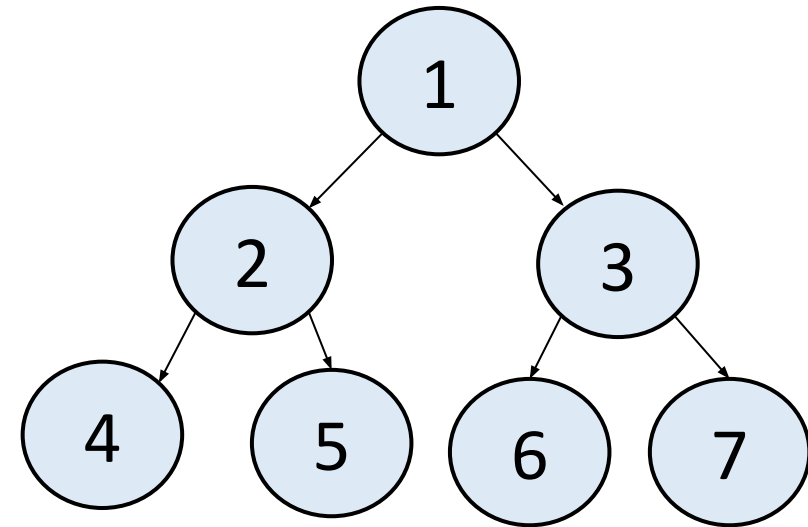
Min heap



Max heap

Implementation

1. Create a complete binary tree.
2. Build Max heap by applying the **Heapify** process.

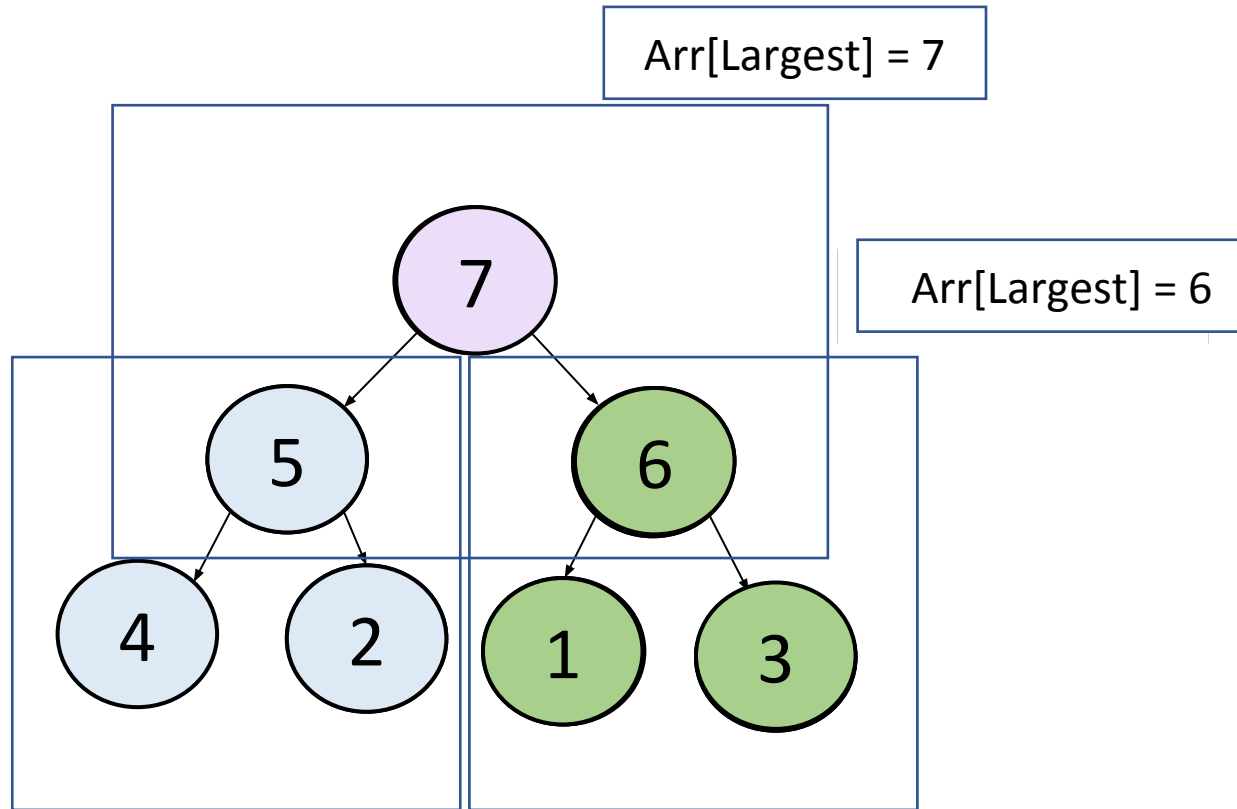


$N = 7$

Leaves start from 3rd node or $N/2$ node

1	2	3	4	5	6	7
---	---	---	---	---	---	---

Heapify



7	5	6	4	2	1	3
---	---	---	---	---	---	---

```
left = 2 * i
right = 2 * i + 1
Largest = index_Largest( Array[i] , Array [left],
Array[right])
if Largest ≠ i:
    swap(Array[i], Array[largest])
    max_heapify(Array, largest, N)
```

build_max_heap(Array)

N = length(Array)

for i from N/2-1 down to 0:

 max_heapify(Array, i, N)

max_heapify(Array, i, N)

left = 2 * i

right = 2 * i + 1

Largest = index_Largest(Array[i] , Array [left], Array[right])

if Largest ≠ i:

 swap(Array[i], Array[largest])

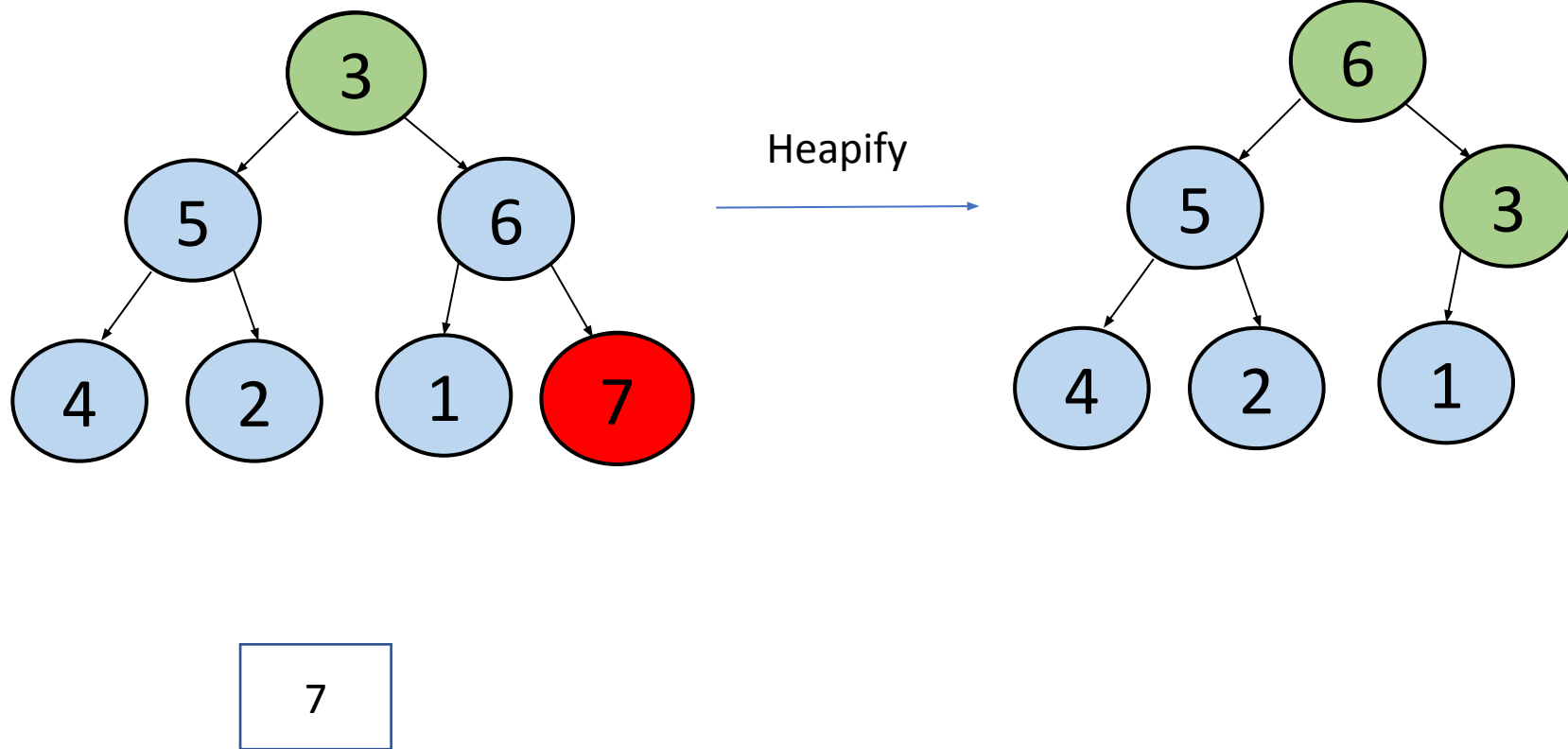
 max_heapify(Array, largest, N)

Heap sort

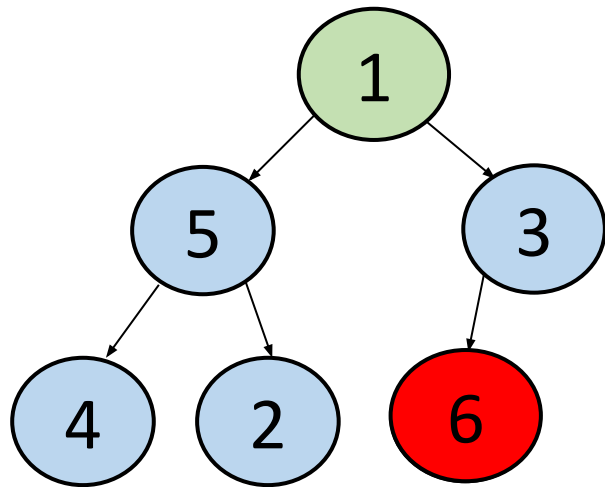
Since the tree satisfies **Max-Heap** property, the largest item is stored at the **root** node.

Removing the root each time returns the data sorted in descending order.

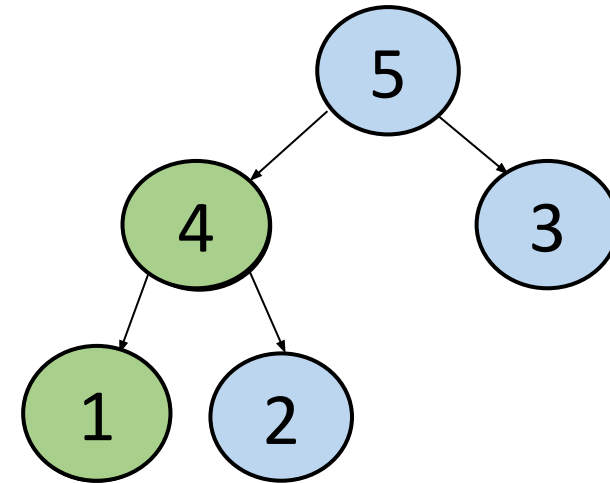
Heap sort



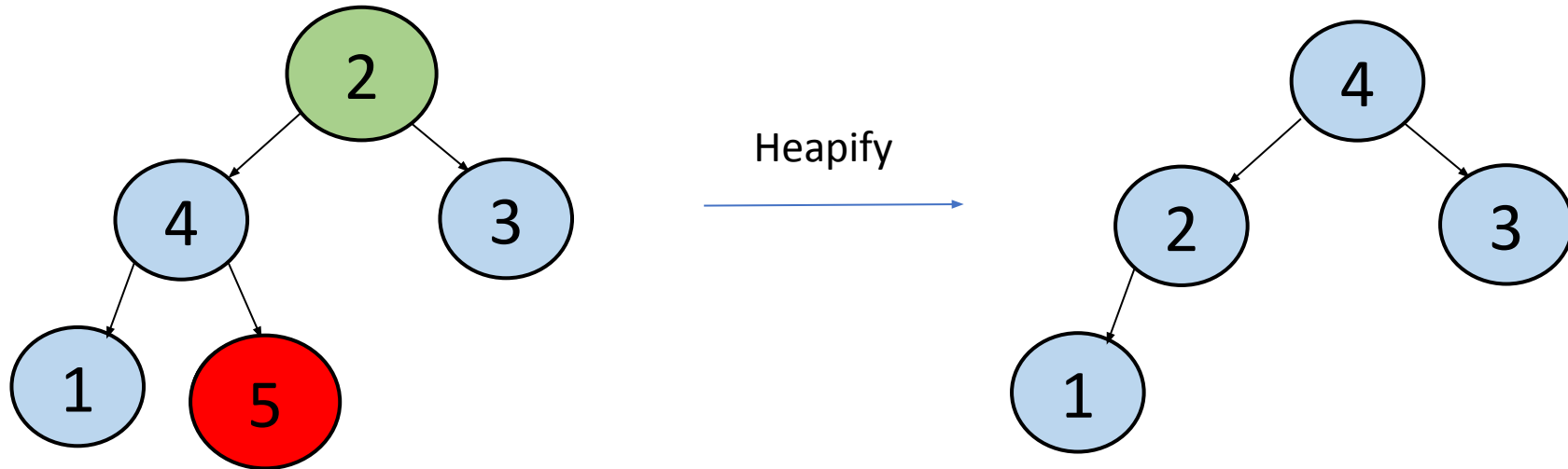
Heap sort



Heapify

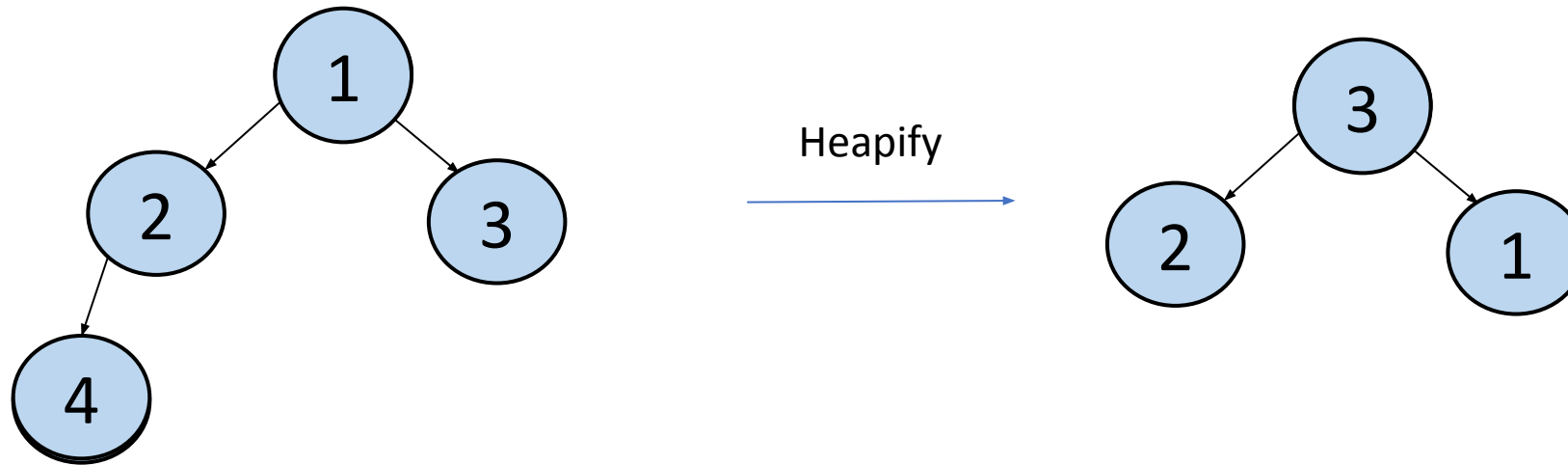


Heap sort



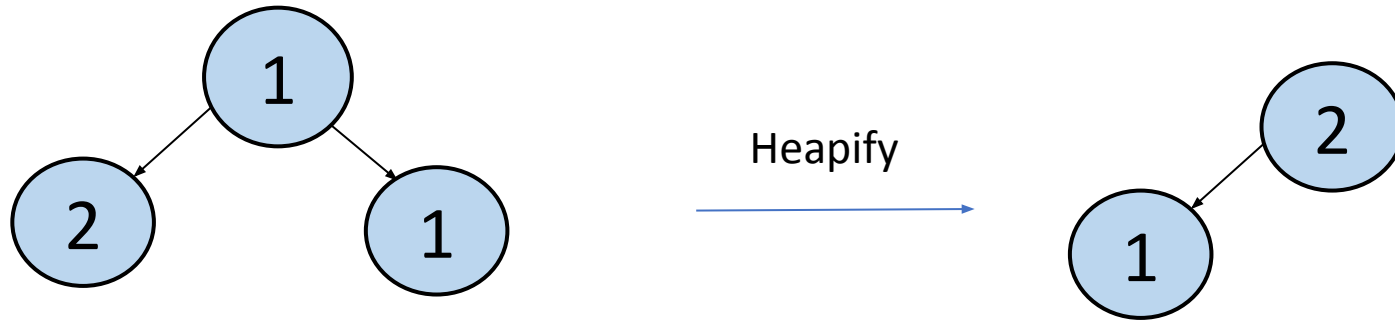
7	6	5
---	---	---

Heap sort

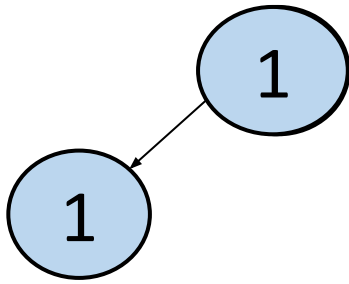


7	6	5	4
---	---	---	---

Heap sort



Heap sort



Heapsort()

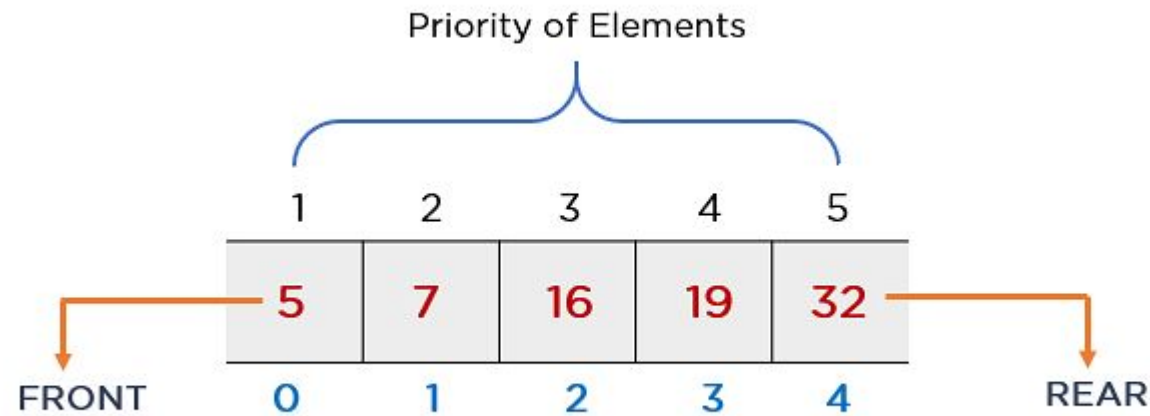
1. Repeat $i=N-1$ to 0:
 1. Swap: Swap the root element and value at the i^{th} position.
 2. Remove: Reduce the size of the heap by 1 by removing the element at i^{th} position.
 3. Heapify: Heapify the tree again with remaining values.

Comparison

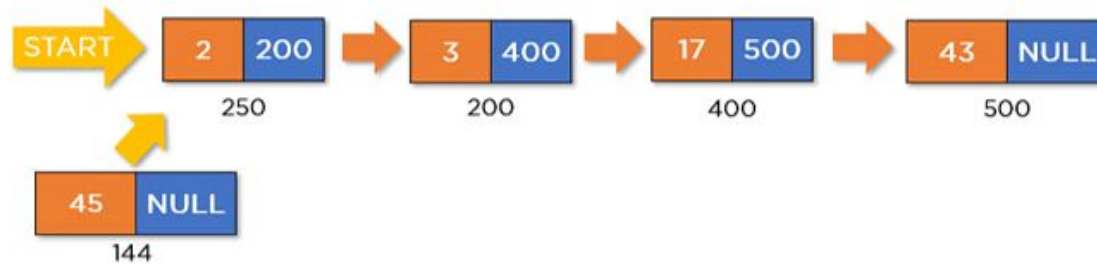
Sorting algorithm	Best Case (Time)	Worst Case (Time)	Average Case (Time)	Worst Case (Space)
Bubble sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Quick sort	$O(n \log(n))$	$O(n^2)$	$O(n \log(n))$	$O(1)$
Heap Sort	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(1)$

Priority Queue

- It behaves similar to a linear queue except for the fact that each element has some priority assigned to it.



Implementation by linked list



Complexities

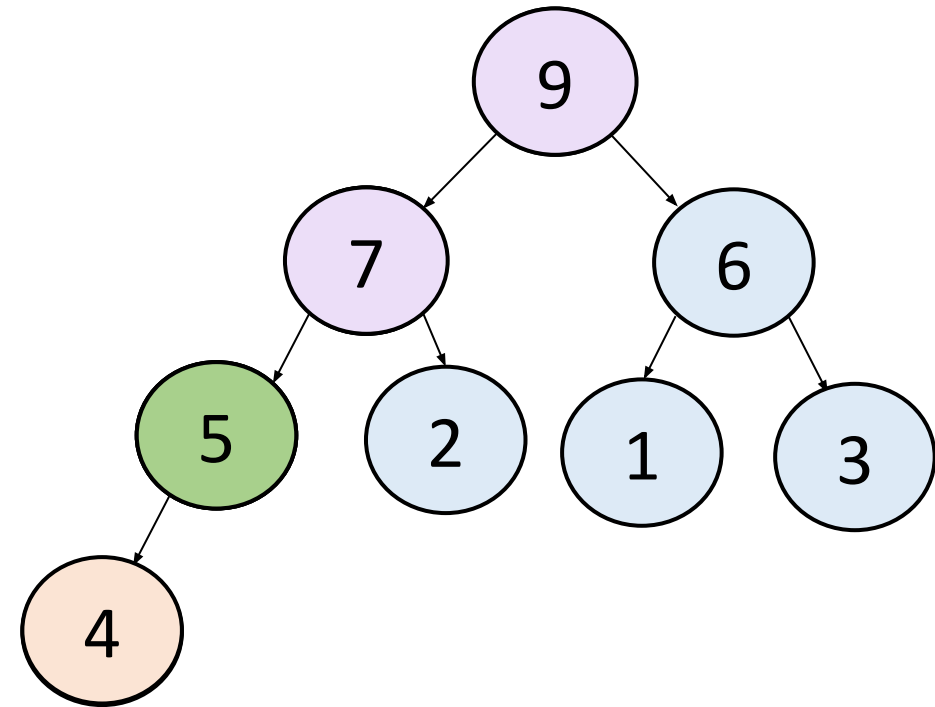
Operations	peek	insert	delete
Linked List	$O(1)$	$O(n)$	$O(1)$
Binary Heap	$O(1)$	$O(\log n)$	$O(\log n)$
Binary Search Tree	$O(1)$	$O(\log n)$	$O(\log n)$

Implementation by Heap

Add new data = 9

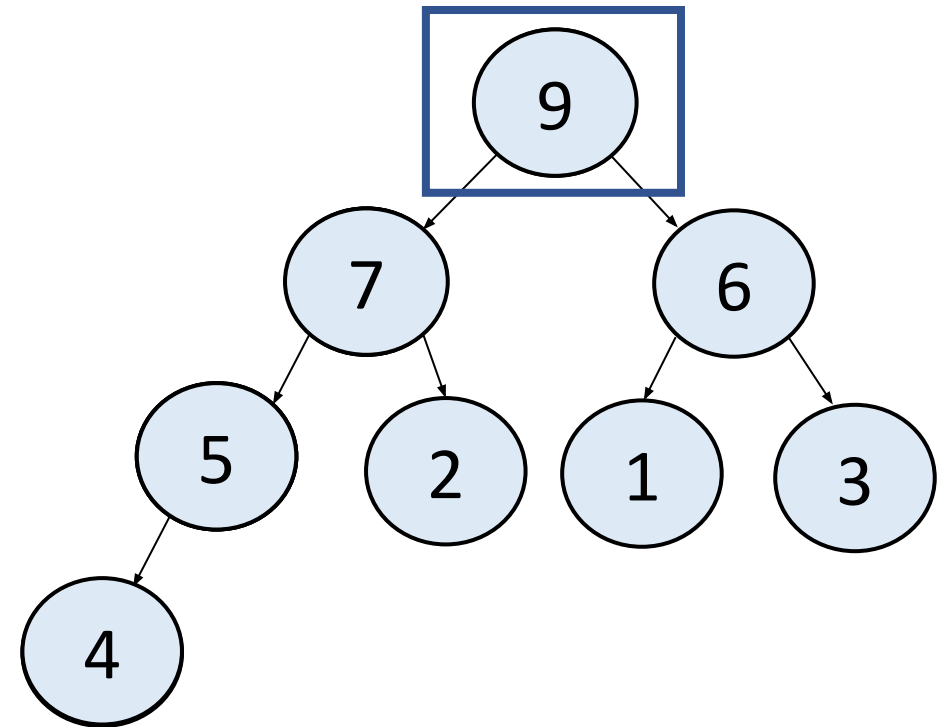
Inserting the Element in a Priority Queue:

1. The new element will move to the Empty space from top to bottom and left to right.
2. Heapify to set the element in the correct position.



Deletion in Priority Queue:

1. Swap: Swap the root element and value at the i^{th} position.
2. Remove: Reduce the size of the heap by 1 by removing the element at the i^{th} position.
3. Heapify: Heapify the tree again with remaining values.



Thank You